

IZVESTAJ O BEZBEDNOSNIM RANJIVOSTIMA

Staticka analiza koda - ProgPilot | 05.05.2026

Automatska staticka analiza izvornog koda PHP aplikacije identifikovala je ukupno **16 bezbednosnih ranjivosti** rasporedjenih u 7 fajlova. Detektovane su ranjivosti tipa Code Injection (RCE), SQL Injection i Cross-Site Scripting (XSS). Dve ranjivosti su ocenjene kao **kriticne** i zahtevaju hitnu akciju.

16
UKUPNO

2
KRITICNE

13
VISOKE

1
SREDNJE

PREGLED RANJIVOSTI

#	Tip	CWE	Ozbiljnost	Fajl	Linija
1	Code Injection (RCE)	CWE-95	KRITICNA	app/admin/import_user.php	7
2	SQL Injection	CWE-89	KRITICNA	app/forgotusername.php	12
3	XSS	CWE-79	VISOKA	app/admin/update_motd.php	41
4	XSS	CWE-79	VISOKA	app/index.php	27
5	XSS	CWE-79	VISOKA	app/index.php	28
6	XSS	CWE-79	VISOKA	app/index.php	29
7	XSS	CWE-79	VISOKA	app/index.php	30
8	XSS	CWE-79	VISOKA	app/index.php	61
9	XSS	CWE-79	VISOKA	app/index.php	80
10	XSS	CWE-79	VISOKA	app/index.php	81
11	XSS	CWE-79	VISOKA	app/index.php	82
12	XSS	CWE-79	VISOKA	app/profile.php	40
13	XSS	CWE-79	VISOKA	app/profile.php	42
14	XSS	CWE-79	VISOKA	app/profile.php	44
15	XSS	CWE-79	VISOKA	app/resetpassword.php	77
16	XSS (payload gen)	CWE-79	SREDNJA	solutions/rce/generateSerializedPayload.php	10

DETALJNI OPIS RANJIVOSTI

#01 Code Injection / Remote Code Execution

KRITICNA CWE-95

Fajl: app/admin/import_user.php

Izvor (source): \$userObj (linija 5)

Ponor (sink): unserialize() (linija 7)

Opis ranjivosti:

Korisnicki kontrolisani podatak (\$userObj) se direktno prosledjuje PHP funkciji **unserialize()** bez prethodne validacije ili sanitizacije. Napadac moze da konstruise maliciozan serializovani objekat koji, pri deserijalizaciji, aktivira PHP magic metode (__destruct, __wakeup) i izvršava proizvoljni kod na serveru.

Potencijalni uticaj:

Potpuno preuzimanje servera, izvršavanje komandi, kradzja podataka, instalacija backdoor-a.

Preporucene mere zastite:

1. Izbegavati unserialize() sa podacima iz nepouzdatih izvora.
2. Koristiti JSON umesto PHP serializacije.
3. Ako je unserialize() neophodan, koristiti allowed_classes opciju.
4. Implementirati HMAC potpisivanje serializovanih podataka.

Referenca: https://owasp.org/www-community/vulnerabilities/PHP_Object_Injection

#02 SQL Injection

KRITICNA CWE-89

Fajl: app/forgotusername.php

Izvor (source): \$username (linija 9)

Ponor (sink): pg_query() (linija 12)

Opis ranjivosti:

Korisnicki unos (\$username) se direktno ugradjuje u SQL upit koji se izvršava funkcijom **pg_query()**. Napadac moze da manipulise SQL upit ubacivanjem specijalnih karaktera i time dobije neautorizovani pristup podacima, zaobidze autentifikaciju ili izvrši destruktivne operacije nad bazom podataka.

Potencijalni uticaj:

Neautorizovani pristup svim podacima u bazi, mogucnost brisanja/izmene podataka, zaobilazenje autentifikacije.

Preporucene mere zastite:

1. Koristiti pripremljene iskaze (prepared statements) sa parametrizovanim upitima.
2. Koristiti pg_prepare() i pg_execute() umesto pg_query() sa konkatencijom.
3. Primeniti princip najmanjeg privilegija na DB korisnika.
4. Validirati i ograniciti ulazne podatke.

Referenca: https://owasp.org/www-community/attacks/SQL_Injection

#03 Cross-Site Scripting (XSS) - Stored/Reflected

VISOKA CWE-79

Fajl:	Vise fajlova (index.php, profile.php, update_motd.php, resetpassword.php)
Izvor (source):	Podaci iz baze (\$row[]), GET parametri, template promenljive
Ponor (sink):	echo / print bez enkodiranja

Opis ranjivosti:

Detektovano je **13 XSS ranjivosti** u vise fajlova. Podaci iz baze podataka i GET parametri se direktno ispisuju putem echo/print bez HTML enkodiranja. Napadac koji moze da unese podatke u bazu ili kontrolise URL parametre moze ubaciti maliciozni JavaScript koji se izvrsava u pretrazivacu zrtve. Napomena: \$htmlentities_return sugerise pokusaj zastite, ali taint analiza ukazuje na moguće propuste u primeni.

Potencijalni uticaj:

Kradzja session kolacica, preuzimanje naloga, defacement, phishing, distribucija malwarea.

Preporucene mere zastite:

1. Uvek koristiti htmlspecialchars(\$var, ENT_QUOTES, 'UTF-8') pre ispisa u HTML kontekstu.
2. Koristiti Content Security Policy (CSP) HTTP zaglavlje.
3. Za \$htmlentities_return - proveriti da li se enkodiranje primenjuje pre ili posle potencijalne manipulacije.
4. Koristiti templating engine sa automatskim enkodiranjem (Twig, Blade).

Referenca: <https://owasp.org/www-community/attacks/xss/>

Ovaj izvestaj je generisan automatski na osnovu staticke analize alata ProgPilot. Staticka analiza moze sadrzati lazne pozitivne - preporucuje se manuelna verifikacija svake ranjivosti. Prioritet otklanjanja: kriticne ranjivosti (RCE, SQLi) moraju biti ispravljene pre produkcijskog deployments.